

Introducing EWE-1

2026-04-15

Large Transaction Models (LTMs)

Foundation models for financial transactions have been gaining traction over the last few year: Featurespace (now part of Visa) launched a large transaction model, [TallierLTM](#), in 2023 to combat fraud and financial crime. In May 2025, Stripe announced that they had developed the [Payments Foundation Model](#) used for fraud detection and dispute resolution, among other applications. In 2026, both [MasterCard](#) and [Revolut](#) announced their transaction foundation models. All of these models share fundamentally the same approach: Tokenize transactions and use a transformer-based architecture to learn a representation from sequences of these tokenized transactions. Important to the usefulness of these representations is the scale of the training data: it typically consists in billions or tens of billions of transactions.

LTMs on Blockchain

This approach has also been applied to blockchain data. The main examples applying a pure sequence modelling approach to blockchain data are [BERT4ETH](#) and [ZipZap](#), and a number of other papers have explored combining graph neural network elements with transformers in various ways. They all aim, however, for scientific contributions to the field, and typically train their models on datasets of up to tens of millions of transactions (ZipZap trains on up to 110 million transactions), while not publishing their model weights. This is extremely useful for the advancement of science, but leaves a lot of work to researchers or companies that want to build on models of this type.

EWE-1

EWE-1 fills this gap, by training on all Ethereum mainnet transactions from 2024 and 2025, excluding only those where neither address had at least 4 incoming or outgoing transactions in that period, and those where both parties had more than 100k transactions over their lifetime, for a total of 1.1 billion transaction records. In addition, the model comes in three sizes with 35, 110 and 500 million parameters, and the model weights are released on [huggingface](#), as is model inference code on [github](#). The weights and the inference code are permissively licensed. **EWE-1** is the first open-weights model of a blockchain-scale large transaction model, and can be used for commercial or research purposes free of charge.

Input features

The **EWE-1** models take 31 features per transaction as input, and ingests the prior 64 incoming and outgoing transactions for a particular ethereum address featurised in this way. If less than 64 prior transactions are available, the sequence is padded with null values to expand it to the required length. The features capture a number of aspects of each transaction: the counterparty, the characteristics of the transaction itself and the history of the wallet at the point in time of the transaction. The immediate and final counterparty are each recorded (distinct if the immediate counterparty is a smart contract), as is the contract itself. The transaction features themselves include things like the transaction cost, the transaction input entropy, the success (or otherwise) of the transaction and the time and date. The address history is represented via address age, transaction frequency, time since the last outgoing transaction was recorded and the number of smart contracts deployed by it. Together, these features capture 1) who the transaction involves 2) how the transaction is executed 3) what the state of the wallet is prior to the transaction. To have a more detailed look at the features used in EWE-1, check out the [full input feature list](#).

Model architecture & training

The **EWE-1** models were trained using [sequifier](#), a framework for training and inferring multivariate causal transformers. This makes the **EWE-1** models *causal* embedding models, which compress the transaction history to a state representation that is maximally informative about future behaviour. This is a notable departure from the more common approach of training BERT-style models using masked reconstruction tasks. In these, the whole sequence is fed in and used to predict values that were masked in the input *across the whole sequence*. This formulation pushes the model to learn to represent sequences as a whole, with an optimisation goal that gives equal importance to early as to late transactions. The EWE-1 model embeddings, by contrast, maximise the informational content of embeddings about *future states*, discarding the information contained in much earlier transactions if doing so helps in its objective.

Within the sequifier architecture for EWE-1, the input variables are projected into separate embedding spaces, through embedding layers for discrete input variables and linear layers for real valued input variables. They are then concatenated, downprojected to the transformer layer size and fed into the transformer. The transformer backbone itself has 16 attention heads for all model variants, while the number of layers and embedding size vary by model variant, and are summarised below.

Model Name	Embedding Dimension	Number of Layers
EWE-1-slim-small	384	12
EWE-1-slim-medium	768	16

Model Name	Embedding Dimension	Number of Layers
EWE-1-slim-large	1536	24

Model Validation

The primary validation metric employed during the development of **EWE-1** was wallet separation on the validation set: how high is the cosine similarity between the embeddings of the same address at two different time points with no overlapping transactions, compared to the cosine similarity between two randomly chosen wallets? On the test set, the within-wallet cosine similarity was typically very high, between 85% and 90%, while the between-wallet cosine similarity was between 10% and 15%. These metrics do not markedly change with model size.

The prime downstream tasks used in the literature are Phishing account detection and deanonymization. Unfortunately, the datasets that are available are severely dated, and typically include addresses mainly from 2017 to 2022. For a model trained exclusively on 2024 and 2025 transactions, these fall largely out of scope. Nonetheless, we ran an analysis on the transactions of the phishing addresses provided [here](#), and a matched set of random ethereum addresses with similar time profiles (first transaction date, last transaction date, number of transactions). The last 64 transactions before the date of the publication of that list were collected for the phishing addresses and the matching dataset, featurized and split into train and test set. Two random forest classifiers were trained to predict whether an address was used for phishing, one from the embeddings created by EWE-1 from the train set data, and one on the train set data directly. The one trained on the embeddings outperformed the model based on raw input features by a couple of percentage points (reducing error rates by 10-20%), but further preprocessing and hyperparameter optimization might improve both approaches significantly. In the final analysis, these numbers cannot be conclusive, and the usefulness of the embeddings must be evaluated for specific use cases using current data.

Further Links

- The models are available on our [huggingface page](#)
- The sequifier framework can be found on [github](#)
- And the EWE-1 family of models also has a [custom website](#)

Addendum

Full input feature list

Counterparty identifiers

- `correspondent_address`: The ethereum address that is the sender or receiver of that transaction

- `final_counterparty`: The final counterparty if the immediate counterparty is a smart contract that in turn transfers tokens to a different address
- `related_contract`: The id of the smart contract, if that is the counterparty
- `method_id`: The selected function

Transaction features:

- `is_sender`: Whether it is an incoming or outgoing transaction
- `is_successful`: If the transaction was successful
- `is_contract_call`: Whether the transaction is a contract call
- `tx_type`: Transaction envelope type
- `tx_cost_eth`: The transaction fee
- `is_self_tx`: Whether the sender and receiver have the same address
- `is_early_block`: Is in the first 3 transactions within the block
- `token_value_log`: Token Value
- `input_len`: The length of the input field
- `input_entropy`: Entropy of the input field

Wallet features:

- `s_age_days`: Account age
- `s_time_since_last`: Time since last transaction
- `s_time_since_send`: Time since last outgoing transaction
- `s_nonce`: Number of total outgoing transactions
- `s_fail_rate`: Share of failed transactions among outgoing transactions
- `s_io_ratio`: The share of total transactions that is incoming, rather than outgoing
- `s_freq_std_dev`: The standard deviation of the intervals between outgoing transactions
- `s_contract_count`: The number of smart contract deployed from this account
- `s_session_depth`: The number of transactions in the same session, including the current one
- `s_is_new_session`: Whether the transaction is the first in a new session (No outgoing transaction in the past 15 minutes)
- `s_inter_arrival_ratio`: The change in transaction speed
- `s_lifetime_erc20`: Number of standard ERC-20 token actions performed by an address
- `s_total_interactions_log`: The log of the total number of incoming and outgoing transactions

Time based features:

- month of the year
- day of the month
- day of the week
- hour of the day

sistemalabs

© 2026 sistemalabs research OÜ. All rights reserved.

[privacy](#) [analytics preferences](#)